

Gestiona bases de datos con MySQL

Día 3

Gerald Kogler

geraldo@servus.at

Instrucciones preliminares

Podéis descargar este documento aquí:

https://github.com/geraldo/curso_mysql_online/blob/main/mysql_dia3.pdf

Posteriormente hace falta descargar el instalador de XAMPP desde aquí:

<https://www.apachefriends.org/download.html>

Seguimos los pasos de instalación de XAMPP para el sistema operativo usado. Si usáis Windows entonces recomiendo instalarlo en la carpeta predefinida:

<c:\xampp>

Por cualquier problema que nos encontramos podemos consultar la ayuda online:

https://www.apachefriends.org/faq_windows.html

Temario día 3

7 Funciones y vistas

7.1 Funciones de MySQL

7.2 Vistas

8 Unions, subqueries y joins de SQL

8.1 Unions

8.2 Subqueries

8.3 Joins

9 Triggers, eventos y índices

9.1 Triggers

9.2 Eventos

10 Mantenimiento de MySQL

10.1 ANALYZE TABLE

10.2 CHECK TABLE

10.3 CHECKSUM TABLE

10.4 Defragmentar tabla

10.5 FLUSH

10.6 OPTIMIZE TABLE

11 Administración de la Base de Datos

11.1 Copias de seguridad y restauración

11.2 Permisos

11.3 Roles

7 – Vistas y funciones

MySQL – SQL Views

Vistas son consultas en forma de tabla. Externamente funcionan idénticamente a las tables. Esto permite hacer tables dinámicas que obtienen los valores de otras tables.

Muy útil para hacer una capa con información y cargarla en una aplicación web (Wordpress) o escritorio (QGIS). Eso tiene el efecto que la información puede estar guardada en diferentes lugares pero la aplicación lo ve como una sola tabla con información.

Una vista siempre muestra la información actualizada de las tablas utilizadas, es decir, cada vez que se actualizan datos en una tabla utilizada por una vista, esta vista se vuelve a crear.

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Igual que las tablas y las bases de datos, podemos borrar una vista usando DROP VIEW:

```
DROP VIEW view_name;
```

MySQL – SQL Views

Creemos una vista de los municipios con más de 10.000 habitantes en la provincia de Girona:

```
CREATE OR REPLACE VIEW `[Municipios grandes en Girona]` AS
```

```
SELECT codimuni, nommuni, habitants
```

```
FROM municipis
```

```
WHERE codiprov=17 AND habitants>10000;
```

Ejercicios:

- Crear una vista de todos los municipios de la comarca del Alt Penedès.
- Crear una vista de todos los municipios de Catalunya en una altura de más de 1000 metros.

MySQL – MySQL Functions

MySQL tiene funciones para tratar cadenas de texto, números y fechas. Aquí trataremos algunos para entender su aplicación, un listado completo de las funciones encuentras en

https://www.w3schools.com/mysql/mysql_ref_functions.asp

Algunas funciones de cadenas de texto:

UPPER() convierte una cadena de texto en mayúsculas:

```
SELECT UPPER("Aprendemos SQL y más!");  
SELECT UPPER(titulo) from cursos;
```

LENGTH() devuelve la cantidad de caracteres de una cadena de texto:

```
SELECT LENGTH("SQL");  
SELECT nombre, LENGTH(nombre) AS caracteres from participantes;
```

CONCAT() junta varias cadenas de texto en una cadena única:

```
SELECT CONCAT(" Aprendemos ", "SQL ", "y ", "más!");  
SELECT CONCAT(tutor, " imparte el curso ", titulo) from cursos;
```

MySQL – MySQL Functions

Algunas funciones de números:

Anteriormente ya vimos las funciones de COUNT(), SUM(), AVG(), MIN() y MAX(), aquí otras funciones de tratamiento de números:

FLOOR() devuelve un número entero:

```
SELECT FLOOR(25.75);
```

ROUND() redondea un número con la cantidad de decimales indicadas:

```
SELECT ROUND(135.375, 2);
```


MySQL – MySQL Functions

Algunas funciones de fechas:

NOW() devuelve la fecha y hora actual:

```
SELECT NOW();
```

DATE_FORMAT() formatea una fecha según el formato indicado:

```
SELECT DATE_FORMAT("2024-12-31", "%Y");
```

```
SELECT DATE_FORMAT(fecha_comienzo, "%W %M %e %Y") FROM cursos;
```

El listado completo de formatos a usar verás en este listado: https://www.w3schools.com/mysql/func_mysql_date_format.asp

MySQL – MySQL Functions

Algunas funciones avanzadas:

IF() permite especificar un condicional:

```
SELECT IF(500>1000, "YES", "NO");  
SELECT titulo, IF(NOT STRCMP(tutor, "Gerald Kogler"), "soy yo", "NO soy yo") FROM  
cursos;
```

CASE() repasa múltiples condiciones y devuelve la primera que se cumpla:

```
SELECT count(*),  
CASE  
    WHEN continent="South America" OR continent="North America" THEN "America"  
    ELSE continent  
END as continent_group  
FROM `countries`  
GROUP BY continent_group;
```

ISNULL() comprueba si un valor es NULL:

```
SELECT ISNULL(NULL);
```

8 – Unions, subqueries y joins de SQL

MySQL – SQL Union

La instrucción UNION une las filas de los resultados de múltiples queries en la primera. Para hacerlo es obligatorio que cada SELECT tiene la misma cantidad de columnas, que cada columna tenga el mismo tipo de datos y que estén en el mismo orden en cada SELECT. Mantendrán el nombre de las columnas de la primera query.

Para aplicar instrucciones al resultado se tiene que usar una subquery.

```
SELECT 1 AS resultat, id FROM tabla 1
UNION
      SELECT 2, id FROM tabla 2
;
```

Ejercicios:

- Listar los nombres de municipios con sus códigos de las tablas municipis y de centres_docents en una sola tabla.

MySQL – SQL Subqueries

Las subqueries permiten utilizar el resultado de una query en otra query (de aquí su nombre). Estas subqueries pueden servir de tablas hijas (si se limita a una fila por cada subquery) o valores (si tienen una hija y una columna).

```
SELECT m.nomprov
FROM municipis as m
WHERE codimuni = (
    SELECT `Codi municipi_6`
    FROM centres_docents
    WHERE BATX="BATX"
    GROUP BY `Codi municipi_6`
    ORDER BY count(*) DESC
    LIMIT 1
);
```

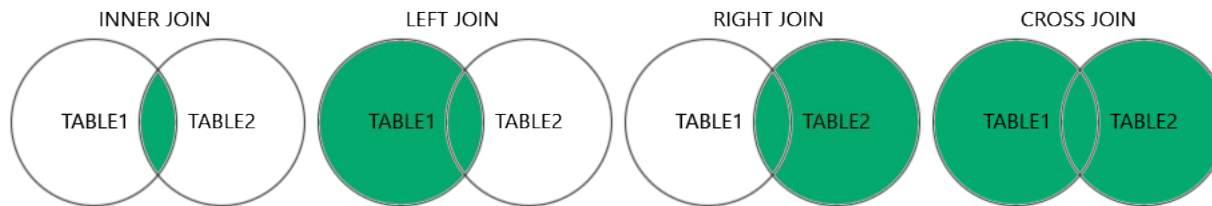
Nota: La subquery no acaba en punto y coma (;) ya que el comando no se ejecuta hasta que la query principal se acaba.

MySQL – SQL JOIN

Importamos el fichero **centres_docents.sql** con todos los centros educativos de Cataluña. Este fichero tiene una columna llamada *codi_municipi_6* que nos permite relacionarlo con la tabla de municipios, que tiene el código del municipio en la columna *codimuni*.

Tipos de JOIN:

- INNER JOIN: Devuelve los registros que tiene valores que cumplan las condiciones en las dos tablas.
- LEFT JOIN: Devuelve todos los registros de la tabla izquierda y solamente los que cumplan las condiciones de la tabla derecha.
- RIGHT JOIN: Devuelve todos los registros de la tabla derecha y solamente los que cumplan las condiciones de la tabla izquierda.
- CROSS JOIN: Devuelve todos los registros de las dos tablas.



MySQL – SQL JOIN

SQL JOIN permite agrupar dos (o más) tablas en una sola en la misma consulta. La condición que agrupa las diferentes filas de las tablas es la condición ON.

```
SELECT m.nommuni, c.`denominació completa`  
FROM municipis AS m  
LEFT JOIN centres_docents AS c  
ON m.codimuni = c.`Codi municipi_6`  
GROUP BY m.nommuni, c.`denominació completa`  
ORDER BY m.nommuni;
```

Como se puede ver en el ejemplo se pueden utilizar alias por las tablas. Esto es muy útil para no tener que reescribir el nombre de la tabla entera cada vez.

MySQL – SQL JOIN

Miremos la diferencia de la misma query usando un INNER JOIN:

```
SELECT m.nommuni, c.`denominació completa`  
FROM municipis AS m  
INNER JOIN centres_docents AS c  
ON m.codimuni = c.`Codi municipi_6`  
GROUP BY m.nommuni, c.`denominació completa`  
ORDER BY m.nommuni;
```

En este caso solamente nos devuelve los municipios que tengan un centro asociado, es decir, no muestra municipios con el centro con valor NULL.

En este caso el resultado devuelve los mismos resultados que haciendo un RIGHT JOIN o un CROSS JOIN ya que todos los centros docentes tienen un municipio asociado.

MySQL – SQL JOIN

Se pueden encadenar múltiples JOIN ... ON ... JOIN ... ON ... para juntar más de dos tables. La manera de como se juntaran dependerá de la relación especificada en ON.

Los joins son muy comunes en los modelos relacionales ya que los datos finales suelen estar en otra tabla (para evitar repeticiones).

```
SELECT *  
  FROM taula1 AS p  
    LEFT JOIN taula2 AS s  
      ON p.t2 = s.id  
    LEFT JOIN taula3 AS t  
      ON s.t3 = t.id  
 WHERE p.id > 100  
 LIMIT 100  
;
```

Ejercicio:

- Mostramos un listado de todos los bachilleratos (BATX) por municipio ordenado por cantidades.
- Mostramos un listado de todos los institutos (ESO) por provincia.

Ejercicios SQL

Para terminar la sesión practicamos algunos comandos SQL que aprendimos. Para eso cargamos un nuevo fichero de datos con consumos energéticos llamado **consums_electrics.sql**.

1. ¿Qué consumo eléctrico tiene el municipio Prat de Llobregat?
2. ¿Cuál es el municipio de Cataluña que tiene más consumo eléctrico industrial?
3. ¿Cuánto consumo eléctrico tiene la industria en la comarca del Baix Llobregat?
4. Agrupa el consumo eléctrico de Cataluña por sectores.
5. Agrupa el consumo eléctrico de la comarca de Barcelona por municipios.
6. Agrupa el consumo eléctrico de la comarca de Barcelona por municipios y sectores.
7. Mostramos un listado de todos los consumos privados por provincia.
8. Mostramos un listado de todos los consumos privados por municipio y habitante.

9 – Triggers y eventos

MySQL – Triggers

Una trigger es un evento que se dispara cuando una condición se cumpla. Cada trigger ejecuta una función que es un poco especial cada vez que una condición se cumpla.

Son muy útiles para automatizar cambios, puede hacer desde calcular alguna columna hasta modificar otras tablas para hacer que los valores sean consistentes.

El trigger se puede activar de dos maneras (*trigger_time*):

- Antes de la instrucción, esto permite modificar los valores que esta aplica (*BEFORE*).
- Después de la instrucción, esto permite utilizar funciones que lean los cambios hechos (*AFTER*).

El trigger se ejecuta antes o después de cada operación insertado (*INSERT*), actualizado (*UPDATE*) o borrado (*DELETE*). Se puede ejecutar una vez por fila (*FOR EACH ROW*) o por el procedimiento completo.

MySQL – Triggers

La sintaxis del trigger tiene la siguiente estructura:

CREATE

```
[DEFINER = user]
TRIGGER [IF NOT EXISTS] trigger_name
trigger_time trigger_event
ON tbl_name FOR EACH ROW
[trigger_order]
trigger_body
```

trigger_time: { BEFORE | AFTER }

trigger_event: { INSERT | UPDATE | DELETE }

trigger_order: { FOLLOWS | PRECEDES } other_trigger_name

MySQL – Triggers

El siguiente ejemplo define un trigger que añade la fecha de creación:

```
CREATE
  TRIGGER last_update
  AFTER UPDATE
  ON cursos
  FOR EACH ROW
  BEGIN
    UPDATE cursos
    SET last_update=NOW();
  END
;
```

Primero tendremos que añadir la columna nueva *last_update* a la tabla *cursos*:

```
ALTER TABLE courses ADD last_update
DATETIME NULL DEFAULT NULL;
```

MySQL – Triggers

Revisamos algunos ejemplos con triggers del manual oficial en <https://dev.mysql.com/doc/refman/8.0/en/trigger-syntax.html>

También puedes consultar este manual muy interesante con casos prácticos: <https://www.mysqltutorial.org/mysql-triggers/>

Ejercicios:

- Crear un Trigger para actualizar la fecha de la última actualización.
- Crear un Trigger para añadir el usuario que ha hecho la última actualización.
- Crear un Trigger para guardar la suma de cursos de un participante (nuevo columna num_cursos).

MySQL – Events

Eventos son acciones que se ejecutarán en un momento específico definido por el evento. Con la siguiente consulta podemos mostrar un listado de eventos activos:

```
SHOW PROCESSLIST;
```

Para crear un evento nuevo usamos la orden **CREATE EVENT** de la siguiente manera:

```
CREATE EVENT [IF NOT EXIST] event_name  
ON SCHEDULE schedule  
DO  
event_body
```

El siguiente evento se ejecutará cada minuto durante una hora y dejará un mensaje en la tabla *messages*:

```
CREATE EVENT recurring_log  
ON SCHEDULE EVERY 1 MINUTE  
STARTS CURRENT_TIMESTAMP  
ENDS CURRENT_TIMESTAMP + INTERVAL 1 HOUR  
DO  
    INSERT INTO messages(message)  
    VALUES(CONCAT('Running at ', NOW()));
```

Para más información consulta este manual: <https://www.mysqltutorial.org/mysql-events/mysql-create-event/>

10 – Mantenimiento de MySQL

MySQL – Analyze Table

ANALYZE TABLE genera estadísticas de una tabla en MySQL.

```
ANALYZE TABLE `consums_electrics`;
```

MySQL – Check Table

CHECK TABLE comprueba que una tabla no tenga errores.

```
CHECK TABLE `consums_electrics`;
```

MySQL – Checksum Table

CHECKSUM TABLE devuelve un checksum, que es un número entero con la cantidad de bytes que ocupa una tabla. Puedes usar esta consulta para verificar que el contenido es exactamente el mismo antes y después de un backup, un rollback, o cualquier otra operación que está pensada para devolver los datos a su estado anterior.

```
CHECKSUM TABLE `consums_electrics`;
```

MySQL – Defragmentar tabla

Insertaciones random o eliminaciones de un índice secundario pueden causar el índice de estar fragmentado.

Fragmentación significa que el orden físico de las páginas índice en el disco duro no es el mismo que el orden de los índices en los registros de las tablas. También puede significar que haya muchos bloques sin usar que han sido alocados al índice.

Un síntoma de la fragmentación es que la tabla ocupa más espacio que debería. Otra síntoma es que un escaneo de la tabla dura mucho más de lo que debería.

```
SELECT COUNT(*) FROM t WHERE non_indexed_column <> 12345;
```

Esta consulta hace un escaneo completo de una tabla de MySQL, lo que es la operación más lenta en una tabla grande.

Para hacer más rápido los escaneos, se puede hacer periódicamente la siguiente operación que rehace la tabla:

```
ALTER TABLE tbl_name ENGINE=INNODB;
```

MySQL – Flush

Hay diferentes tipos de FLUSH de una tabla, lo que necesitan diferentes tipos de permisos. Se puede borrar y rehacer diferentes caches internos y también soltar bloqueos.

```
FLUSH TABLE `consums_electrics`;
```

MySQL – Optimize Table

OPTIMIZE TABLE reorganiza la memoria física de una tabla y sus índices. Eso reduce el espacio usado y mejora las operaciones de entrada y salida (I/O), es decir, cualquier consulta a la tabla. Los cambios aplicados dependen de la storage engine (en muchos casos es InnoDB, más info en https://dev.mysql.com/doc/refman/8.0/en/glossary.html#glos_storage_engine) utilizadas por la tabla.

```
OPTIMIZE TABLE `consums_electrics`;
```

11 – Administración de MySQL

MySQL – Backup and Restore

Hay diferentes formas de hacer backups de MySQL, dependiendo del cliente utilizado:

Una forma fácil y eficiente es usar Export y Import de phpMyAdmin, en este caso es recomendable utilizar el formato SQL ya que así guardamos también la estructura y los parámetros de las tablas de MySQL.



Exporting rows from "courses" table

Export templates:

New template:

Existing templates:

Template:

Export method:

☒ Quick - display only the minimal options

☐ Custom - display all possible options

Format:



Rows:

☐ Dump some row(s)

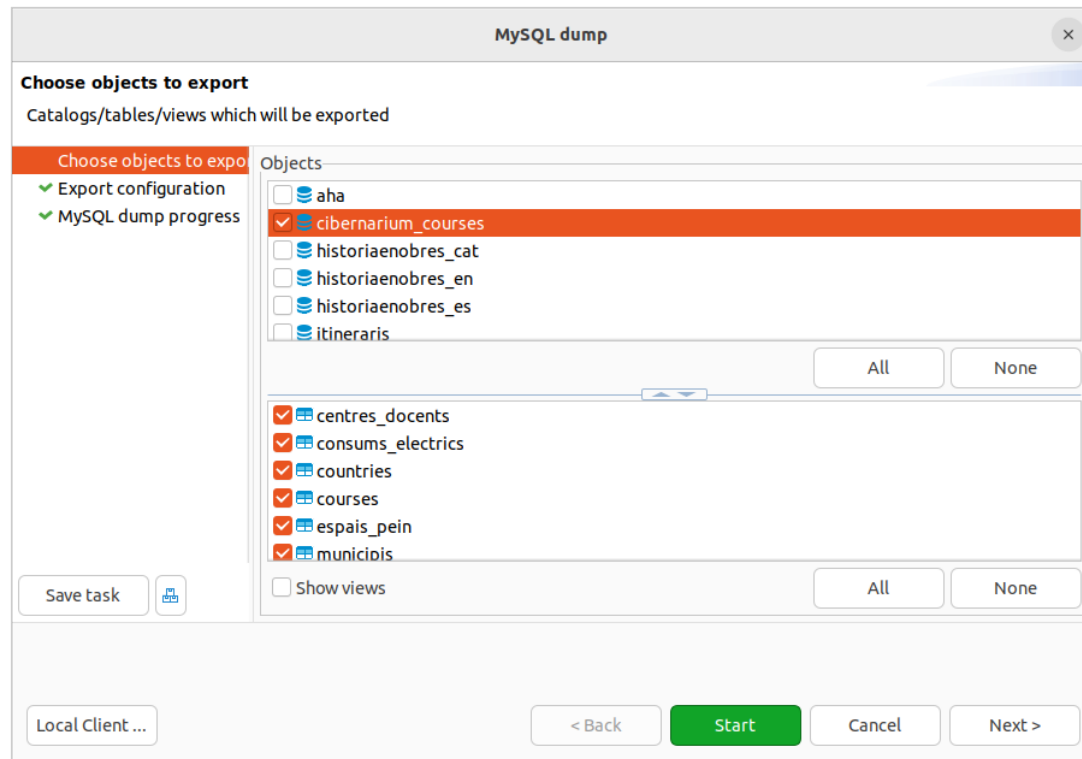
Number of rows:

Row to begin at:

☒ Dump all rows

MySQL – Backup and Restore

La herramienta **mysqldump** está incluido en algunos clientes como MySQL Shell y DBeaver y ofrece más opciones que un simple export de phpMyAdmin.



Más información: <https://dev.mysql.com/doc/refman/8.4/en/mysqldump.html>

MySQL – Permisos y roles

Por definición ya vienen algunos usuarios para gestionar MySQL. Por lo menos siempre va a existir un usuario de administración que tenga todos los permisos sobre las bases de datos, en nuestro caso llamado *root*:

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#) [Operations](#) [Tracking](#) [Triggers](#)

 **Users having access to "cibernarium_courses.courses"**

	User name	Host name	Type	Privileges	Grant	Action
☐	debian-sys-maint	localhost	global	ALL PRIVILEGES	Yes	 Edit privileges  Export
☐	mysql.infoschema	localhost	global	SELECT	No	 Edit privileges  Export
☐	root	localhost	global	ALL PRIVILEGES	Yes	 Edit privileges  Export

 ☐ [Check all](#) With selected:  [Export](#)

MySQL – Permisos y roles

Podemos crear un usuario que solamente tenga derechos de visualizar los datos, pero no modificarlos:

Server: localhost:3306

Databases SQL Status User accounts Export Import Settings Binary log Replication

Add user account

Login Information

User name: Use text field ▼ visor

Host name: Any host ▼ % ⓘ

Password: Use text field ▼ Strength:

Re-type:

Authentication plugin: Caching sha2 authentication ▼

Generate password:

Database for user account

- ☐ Create database with same name and grant all privileges.
- ☐ Grant all privileges on wildcard name (username_%).
- ☐ Grant all privileges on database cibernarium_courses.

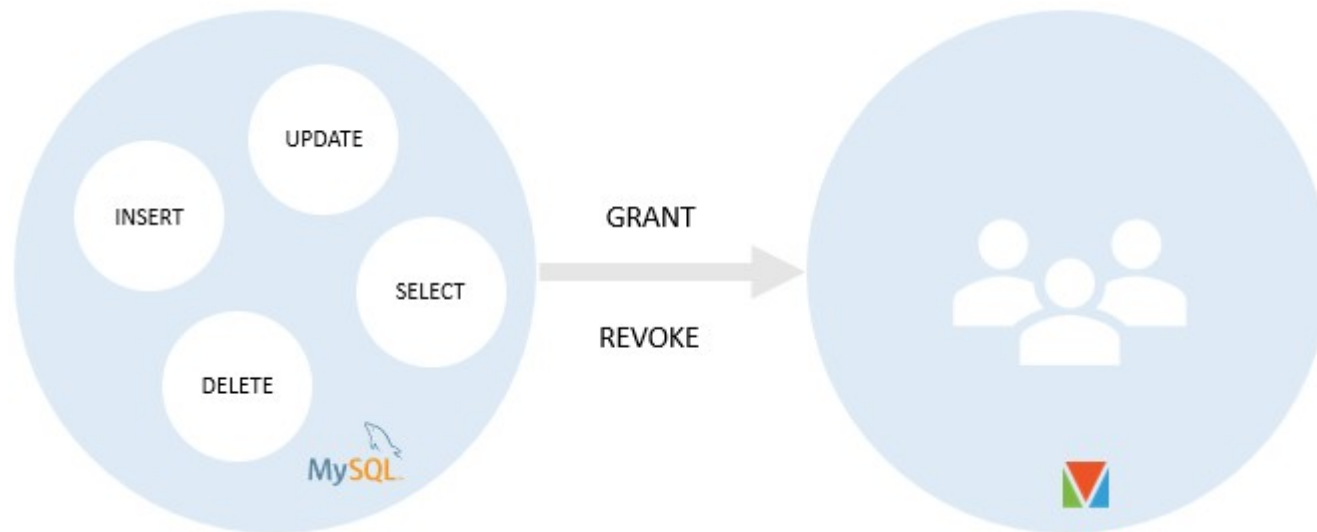
Global privileges ☒ Check all

Note: MySQL privilege names are expressed in English.

☒ Data	☐ Structure	☐ Administration	☐ Resource limits
☒ SELECT ☐ INSERT ☐ UPDATE ☐ DELETE ☐ FILE	☐ CREATE ☐ ALTER ☐ INDEX ☐ DROP ☐ CREATE TEMPORARY TABLES	☐ GRANT ☐ SUPER ☐ PROCESS ☐ RELOAD ☐ SHUTDOWN	<i>Note: Setting these options to 0 (zero) removes the limit.</i> MAX QUERIES PER HOUR 0 MAX UPDATES PER HOUR 0

MySQL – Permisos y roles

Existe el concepto de los roles para facilitar el proceso de gestionar los permisos de los usuarios. Eso permite dar y quitar los mismo permisos a multiples usuarios dentro de un mismo rol:



Aquí un tutorial que explica la gestión de los permisos y roles con SQL: <https://www.mysqltutorial.org/mysql-administration/mysql-roles/>

Espero que os ha agradado esta sesión.
Muchas gracias por vuestra asistencia.

Gerald Kogler

geraldo@servus.at

<https://geraldo.github.io>

<https://github.com/geraldo>

<https://gitlab.com/geraldo1>